Hoofdstuk 3: Zoekalgoritmes

Introductie

Zoeken is fundamenteel voor probleemoplossende strategieën. Als de agent alle informatie over het probleem kent:

1. Formuleer een doel
2. Formuleer het probleem
3. Zoek een oplossing
4. Voer de beste strategie uit

Een zoekprobleem heeft:

- Een eindige **state space**
- Een **initiële state**

Volledig Observeerbaar, Deterministisch

In een deterministisch, volledig observeerbaar probleem:

- Oplossing = een reeks handelingen in de juiste volgorde (**pad**)
- Een pad begint in de begin state en eindigt in de goal state
- **Optimale oplossing** = laagste totale actiekost

Voorbeeld: Antwerpen naar Parijs

We versimpelen het probleem tot een graaf van steden.

```
# States: steden
# Acties: verplaatsen van de ene stad naar de andere
# Transitiemodel: graaf met verbindingen
# Kostfunctie: afstand, reistijd, CO2-uitstoot...

# Representatie in Python
class State:
    def __init__(self, name):
        self.name = name

class Node:
    def __init__(self, state):
        self.state = state
        self.actions = []

    def add_action(self, action):
        self.actions.append(action)
```

Soorten Zoekstrategieën

Type	Kenmerk
Ongeïnformeerd	Kent enkel de graaf, algemeen werkend
Geïnformeerd	Kent ook heuristieken, sneller maar specifieker

Breadth-First Search (BFS)

BFS onderzoekt de search tree niveau per niveau (eerst alle buren, dan hun buren...).

```
from collections import deque

def breadth_first_search(initial_node, goal_state):
    frontier = deque([initial_node])
    explored = set()

    while frontier:
        node = frontier.popleft()
        if node.state.name == goal_state.name:
            return node
        explored.add(node.state)
        for action in node.actions:
            if action.state not in explored:
                frontier.append(action)
    return None
```

Eigenschappen:

Depth-First Search (DFS)

DFS onderzoekt eerst de diepste node in de frontier.

```
def depth_first_search(initial_node, goal_state):
    explored = set()
    return dfs_recursive(initial_node, goal_state, explored)

def dfs_recursive(node, goal_state, explored):
    if node.state.name == goal_state.name:
        return node
    explored.add(node.state)
    for action in node.actions:
        if action.state not in explored:
            result = dfs_recursive(action, goal_state, explored)
            if result:
                return result
    return None
```

Eigenschappen:

Geïnformeerde Zoekalgoritmes: Best-First

Best-first search gebruikt een **heuristiek** om te bepalen welke node het meest veelbelovend is.

Voorbeeld: voor het Antwerpen-Parijs probleem is "vogelvluchtafstand tot Parijs" een goede heuristiek.

Toepassing: Schuifpuzzel

Een schuifpuzzel (8-puzzle, 15-puzzle) kan worden opgelost met zoekalgoritmes:

- **State**: configuratie van de puzzel
- **Acties**: schuiven van een vakje
- **Doel**: opgeloste configuratie
- De search tree bevat alle mogelijke configuraties